

INTER-IIT TECH MEET 14.0

**HIGH PREP PROBLEM STATEMENT
AGENTIC AI**

END TERM REPORT

SUBMITTED BY - TEAM - 11

INDEX

S.No	Title	Page No.
1.	Introduction	3
2.	Uniqueness	3
3.	Problem Understanding & Use-Case Validation	3
4.	Solution Overview	4
5.	System Architecture	4
6.	Results and Metrics	6
7.	Implementation Details & Production Bar	7
8.	Challenges Faced and Solutions	7
9..	User Interface: The Mobile App + The on-device AI Agent	8
10.	Lessons Learned	8
11.	Conclusion	9
12.	References	9

Introduction:-

This project builds a real-time AI Investment Decision system using Pathway to solve the high latency and cost problems of traditional batch-oriented financial systems, which cause delayed insights and missed opportunities for retail investors.

Our production-grade solution ingests live data streams, analyzes them continuously, and generates immediate, personalized investment tips. It takes the user's portfolio, predicts price movements, and provides buy/sell recommendations using real-time stock and news data streaming, Pathway's incremental updates and stateful computations to stay current without expensive recomputations, and pre-computed deterministic components like technical analysis to reduce LLM load. (See Appendix 1.6)

Uniqueness:-

The Real Time AI Investment Decision system differentiates itself from existing solutions through several key novel aspects:

- **Unified, Real-Time Pipeline:** It offers a single, coherent pipeline that combines technical analysis, fundamental analysis, sentiment analysis, and time series models for a comprehensive market view.
- **Alerts and Personalized, Actionable Tips:** The system goes beyond raw charts and metrics by providing personalized tips, including a suggested buy/sell range and potential profit/loss, all tailored to the user's specific portfolio. The tips are sent to the user only when the system expects a sudden change in the stock.
- **Explainable AI Reasoning:** An integrated LLM agent is used to analyze conflicting signals and generate natural language reasoning for every suggested tip, enhancing user trust and understanding.

Problem Understanding & Use-Case Validation:-

We're building a real-time AI investment decision system that processes live stock market and news data to generate actionable investment tips for users. The core problem with traditional approaches is that they process financial data in batches, leading to high latency and costs—meaning users get delayed insights that could result in missed opportunities or losses. By using Pathway's incremental computation framework, we can update metrics in real-time and store them efficiently in ClickHouse, giving users up-to-date predictions on price movements and buy/sell recommendations based on technical analysis, ML models, and sentiment analysis. This matters because most trading apps only show raw charts without real-time analysis or predictions, and retail investors struggle to process the overwhelming amount of stock data, news, and indicators quickly enough to make good decisions.

Recent platforms like QuantConnect, Alpaca, and Bloomberg Terminal have started using machine learning and NLP for trading insights, with frameworks like FinRL and models like ARMA, GARCH, and XGBoost becoming industry standards for predictions. However, these systems still face major challenges with real-time processing, scalability, and explainability—they mostly run in batch mode, which works for long-term predictions but fails to react quickly to rapid market movements. That's exactly why we chose Pathway: it enables us to build a truly real-time, efficient, and intelligent system that processes data as it arrives, computes incremental updates without reprocessing everything, and delivers insights fast enough to actually matter in real-world trading scenarios.

(See Appendix 1.1)

Solution Overview:-

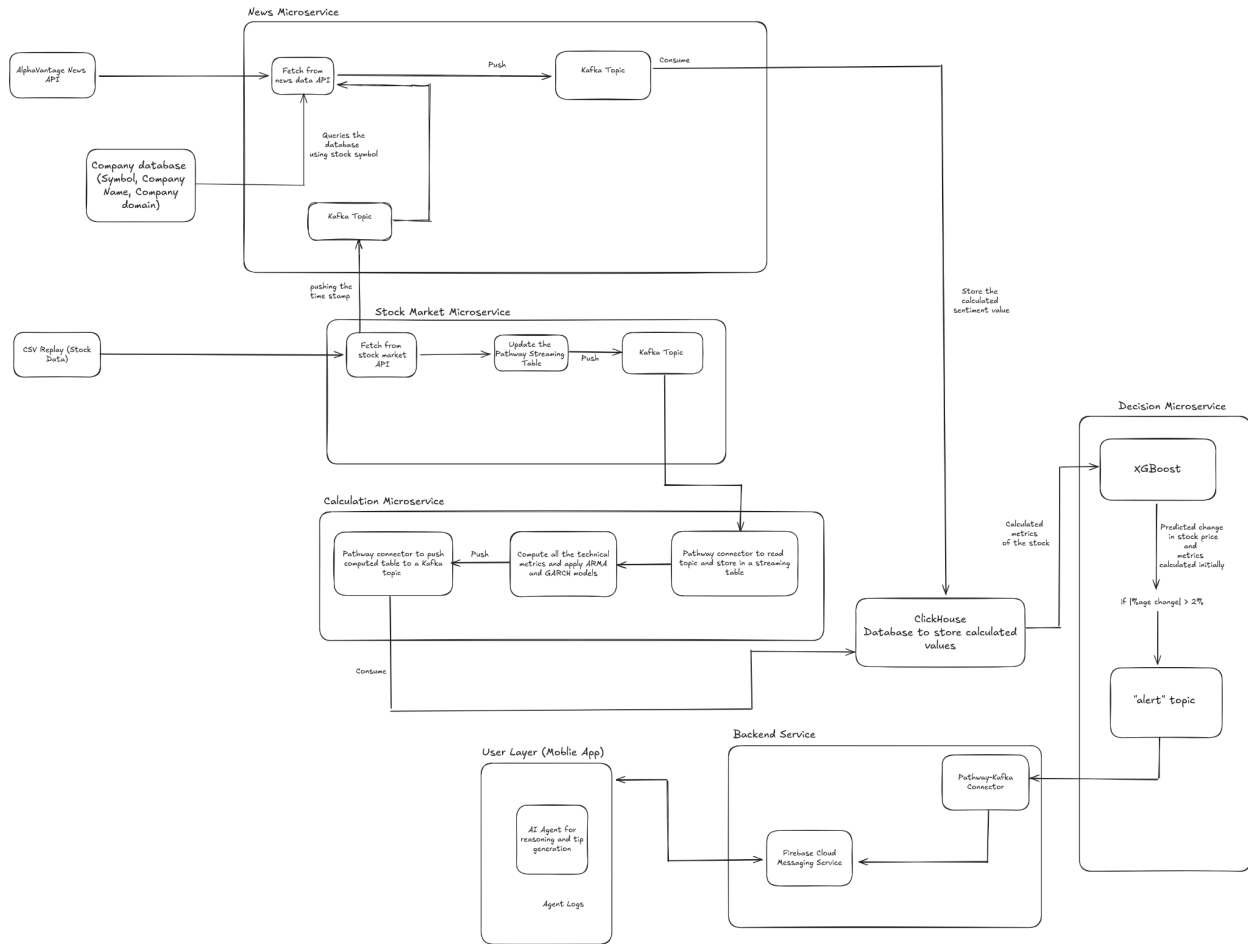
Our system takes a user's portfolio, predicts future price movements for their stocks, and sends alerts with buy/sell recommendations whenever significant changes are expected. It also incorporates real-time news data to improve predictions. Streaming is critical here because users need to act quickly to maximize gains and minimize losses—batch processing would make our tips too late to be useful. Pathway handles the heavy lifting through incremental updates on new data, pre-computing deterministic calculations to reduce LLM load, and providing persistence so we can recover from system failures without losing progress.

The system combines deterministic components like technical analysis (RSI, MACD, etc.), time series forecasters (ARMA, GARCH), and an XGBoost ML model to predict trends. An LLM agent then interprets these results, resolves conflicting signals, and generates clear, natural language tips with reasoning for users. The pipeline works in stages: first, we stream real-time stock prices and news from APIs; second, we calculate technical metrics and apply time series models; third, we feed everything into XGBoost to predict price changes; finally, we generate actionable buy/sell alerts whenever significant movements are anticipated. (See Appendix 1.2)

System Architecture

You can view the architecture diagram using this link:-

<https://excalidraw.com/#json=Kn1KmAxpWR0aybp53kacL,fTkFBmzQ8Y3vxqd05O9Miw>



1. Ingestion of data:- We continuously ingest data of 240 most popular and traded stocks of the US market into our backend through Kafka. **Pathway** consumes these **Kafka topics** through its **Kafka connector**. Also we would be ingesting the real time news related to these companies, as well as their sentiment and relevance score, through an API into Kafka.
2. Technical and Sentiment Analysis:- For each of the stocks present in our list, Pathway calculates the technical metrics. The list of technical metrics calculated are present in Appendix 1.5

The metrics are computed as incremental transformations on Pathway streaming tables for low latency. Pathway's key capabilities—live indexing, incremental updates, materialized state tables with persistence, and low-latency joins and windowing—enable instant signal updates as new data arrives, ensuring sub-second response times that eliminate costly batch solutions used by competitors with Spark or Flink. These metrics help decide when to buy or sell based on momentum and trend alignment. Pathway also processes the news stream and stores it in ClickHouse, contributing to trading decisions.

3. Models used:- We use ARMA models to forecast future values and GARCH models to predict volatility for each stock, with both retrained daily but able to make instant real-time predictions. To capture non-linearity and handle numerous features, we employ two XGBoost models trained on historical price data and technical metrics across all stocks. The first predicts percentage price change, while the second identifies significant moves (changes >2%). Both XGBoost models retrain every few hours. Our key novelty is combining streaming technical indicators, sentiment analysis, incremental ML, and LLM reasoning in a single real-time pipeline.

All computation pipelines run continuously in the backend, minimizing latency when a user requests advice

4. Data Handling:- Once all the metrics are calculated, they would be pushed to **ClickHouse** in one go. ClickHouse will store the historical data which will be used to retrain ARMA, GARCH and XGBoost on the past data.
5. User Interaction and Recommendation Layer:- Once the user has entered his/her current portfolio, as well as investment preferences, our system will send alerts to the user mobile whenever a big change in the stock price is expected. The computed metrics and news of that particular time will be sent to an AI agent, to formulate a user-friendly tip, as well as give proper reasoning for our results. The tip will include at what price should the user buy/sell the stock, or what is the risk score of that trade.

Pathway uniquely enables us: Incremental streaming computation (No recomputation, ultra-low latency), stateful joins (Unifies various tables in real-time) and automatic incremental materialization

Results and Metrics

XGBoost Regressor:- MSE: 0.2007 **R2 Score:** 0.4571

XGBoost Classifier:- Accuracy: 0.9053638537751965

	precision	recall	f1-score	support
0	0.95	0.94	0.95	15851
1	0.51	0.58	0.54	1711
accuracy			0.91	17562
macro avg	0.73	0.76	0.75	17562
weighted avg	0.91	0.91	0.91	17562

Confusion Matrix:- $\begin{bmatrix} 14908 & 943 \\ 719 & 992 \end{bmatrix}$

Implementation Details & Production Bar

Streaming Ingest & Live Indexing

We built our system to handle real-time financial data with low latency while staying reliable. Stock ticks and news continuously flow through lightweight producers into Kafka topics, where Pathway's connectors convert them into dynamic tables. Every new tick triggers incremental updates to technical indicators, sentiment scores, ARMA/GARCH forecasts, and XGBoost feature vectors—all in under a second without recalculating entire windows. We then batch computed features periodically and write them to ClickHouse, which serves as both our historical store and ML training source, with its columnar design enabling fast reads/writes and efficient backtesting.

Resilience & Observability

We ensure expensive operations like model inference or LLM reasoning only run after basic computations are validated, reducing cascading failures. For monitoring, we implemented structured logging through Python and Pathway to track ingestion status, latency, and failures. Before deployment, we validate everything through historical data simulations, schema validation on Kafka payloads to catch malformed data, and safety checks that convert low-confidence signals to "Hold" recommendations. (See Appendix 1.4)

Challenges Faced and Solutions

The challenges faced and their solutions are:

1. Real time computation of metrics for high-volume stock data:- Due to the large number of windows for each of the 240 stock symbols, excessive compute power and memory was being used, which made the system hang. In order to solve this, we optimized the number of sliding windows used for the various calculations, to reduce memory usage.
2. Synchronizing data from stock ticks and news stream:- The news data is fetched from the API every 2 hours, but the stock data is updated every 5 mins. This led to an issue of synchronizing timestamps of the two streams, which was essential for inferring from XGBoost. We solved this by fetching the news data from the API by mentioning the start and end datetime.

3. ML output alone lacks clarity:- The output of the ML model alone may not be sufficient for the user to trust upon, especially in financial decisions. Thus we added a LLM reasoning layer to understand the independent raw signals, and explain the reasoning behind the suggested tip, in user-friendly language.
4. Reducing user screen time through smart alerts:- If users have to constantly monitor dashboards for potential price swings, they lose time and the system becomes inconvenient, defeating the purpose of intelligent automation. Thus we introduced event-triggered alerting, where notifications are sent only when the system predicts a significant expected price movement or risk event.

User Interface: The Mobile App + The on-device AI Agent

The mobile app (still under development) lets users access personalized alerts. During onboarding, users submit their portfolio details and preferences. When the backend sends alert data via Firebase Cloud Messaging, the OS wakes the app in the background and an on-device AI agent processes the user's preferences, portfolio, and received data—but doesn't display the notification yet.

The AI agent only generates and shows a notification if the stock either matches user preferences or could significantly impact their portfolio financially. The agent creates actionable suggestions (like "SELL AAPL at \$x" or "BUY NVDA at \$y") with reasoning based on news headlines. The home screen displays suggestions from the last 12 hours so users can act on any they missed. Users can also update their portfolio and preferences anytime. All data—portfolio, preferences, and recent tips—is stored locally in the device's internal storage (SQLite for Android). (See Appendix 1.3)

Lessons Learned

Working on this project taught us a lot about building real-time systems, ML engineering, and solving practical problems. We gained hands-on experience creating production-ready streaming systems and turning complex financial analysis into something people can actually use. Here's what we learned along the way:

1. **Real-Time Processing Makes a Huge Difference** Traditional batch processing doesn't work in finance where decisions happen in seconds. Using Pathway for incremental computations showed us how much faster and cheaper stateful streaming is than rerunning everything from scratch.
2. **Real-World Data Is Messy** Stock data and news feeds don't play nicely together—misaligned timestamps, missing values, different update rates, and out-of-order arrivals. This taught us the importance of buffers, graceful failure handling, and solid data validation.

3. **Predictions Alone Aren't Enough** Just giving users predictions without reasoning didn't work well. Adding the LLM layer to explain recommendations significantly improved trust and usability, showing us how critical explainability is for financial decisions.
4. **User Experience Is Everything** We initially thought users would monitor dashboards constantly. Smart alerts taught us that convenience and workflow integration matter more than fancy algorithms—great tech is useless if it's hard to use.
5. **Breaking Things Into Modules Pays Off** Separating data ingestion, computation, and recommendations made development smoother. We could work in parallel, debug easily, and set ourselves up for future features like options trading or forex support.

Conclusion

Through this project, we built a real-time AI system that processes live stock market data and news to help with investment decisions. The system calculates financial indicators on the fly, forecasts price movements, and generates personalized, easy-to-understand trading recommendations.

We used Pathway's streaming engine with Kafka for data ingestion and ClickHouse for storage and model retraining. The system combines technical analysis, sentiment analysis, ARMA/GARCH forecasting, and XGBoost for predictions, with an LLM layer that translates raw signals into plain-English tips anyone can understand. The result is a scalable platform that connects streaming data engineering with machine learning and explainable AI—a real-time solution that gives us a clear advantage over traditional batch-processing systems and demonstrates Pathway's power for high-impact financial applications.

References

Pathway Documentation - Pathway, 2024. <https://pathway.com/docs/>

Kafka Documentation – Apache Kafka, 2024. <https://kafka.apache.org/documentation/>

ClickHouse Documentation – ClickHouse Inc., 2024. <https://clickhouse.com/docs/>

XGBoost: Scalable and Portable Gradient Boosting Framework- Chen, Tianqi & Carlos Guestrin (2016). Proceedings of KDD.

ARMA & GARCH Models for Time Series Forecasting- Engle, Robert (1982). *Autoregressive Conditional Heteroskedasticity*. Box, George & Jenkins, Gwilym (1970). *Time Series Analysis: Forecasting and Control*.